

Computer Vision Lab 02

Image Classification using Convolutional Neural Network (CNN)

Lab Objectives

- Understand image classification problem
- Learn basics of Convolutional Neural Networks
- Train a CNN on a small image dataset
- Evaluate model performance using accuracy

Software & Tools Required

- Python 3.8+
- Google Colab or Local Python Environment
- PyTorch and Torchvision
- PIL Library

Dataset Description

This lab uses a simple synthetic dataset with two classes:
Circle and Square images.

Total Images: 100

Image Size: 64×64

Color Mode: Grayscale

Folder Structure

```
simple_cv_dataset/  
├── train/  
│   ├── circle/  
│   └── square/  
└── test/  
    ├── circle/  
    └── square/
```

Step 1: Import Required Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

Step 2: Data Preprocessing

Convert images to grayscale and tensors.

```
transform = transforms.Compose([
    transforms.Grayscale(),
    transforms.ToTensor()
])
```

Step 3: Load Dataset

```
train_data = datasets.ImageFolder("simple_cv_dataset/train", transform=transform)
test_data = datasets.ImageFolder("simple_cv_dataset/test", transform=transform)

train_loader = DataLoader(train_data, batch_size=8, shuffle=True)
test_loader = DataLoader(test_data, batch_size=8)
```

Step 4: Define CNN Model

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 8, kernel_size=3)
        self.fc = nn.Linear(8*62*62, 2)

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```

Step 5: Training the Model

```
model = SimpleCNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(5):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Step 6: Model Evaluation

```
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print("Accuracy:", 100 * correct / total, "%")
```

Expected Output

- Training loss decreases over epochs
- Accuracy above 90% on test data

Student Tasks

1. Change batch size and observe results
2. Increase number of epochs
3. Add another shape class
4. Compare CNN with simple neural network

Conclusion

This lab demonstrates how CNNs process images and perform classification. It forms the foundation for advanced computer vision tasks.